

AI Banking System — End-to-End RAG Framework, Cloud Pipeline & VM Migration Guide

⚡ NEXT-GEN AI BANKING PLATFORM

AI Banking System

Multi-User Statement Analytics • Multilingual AI Advisor • Smart Loan Engine

PDF / Excel / OCR

10+ Languages

Multi-Currency

Gemini AI Chat

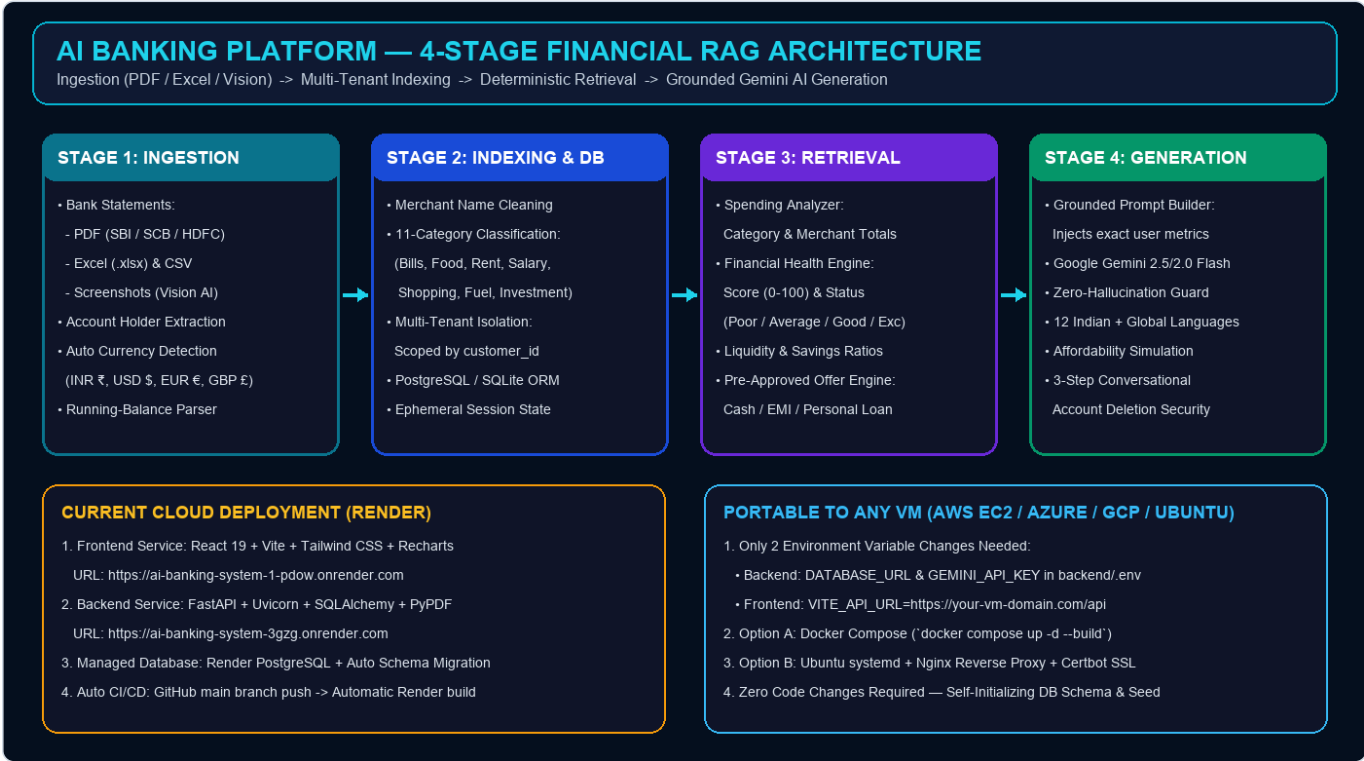
LIVE DEMO & USER ANALYTICS

\$20,000.00 / ₹ INR 80/100 AI

Downloadable Word Document (.docx): - Repository Path: docs/AI_Banking_RAG_Framework_Documentation.docx - Live Cloud Download URL: https://ai-banking-system-1-pdow.onrender.com/AI_Banking_RAG_Framework_Documentation.docx

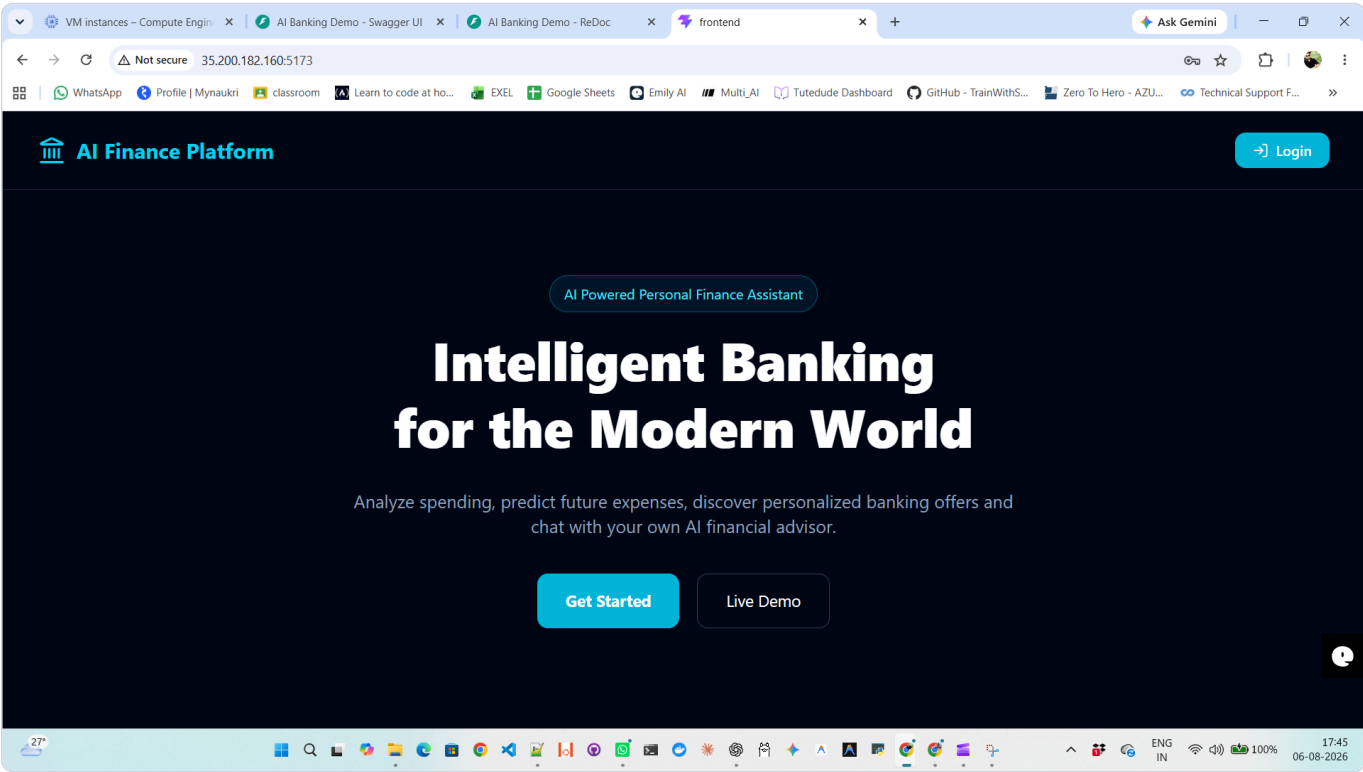
1. Executive Summary & System Overview

The **AI Banking System** is a full-stack intelligent personal finance, expenditure analytics, and wealth management platform built around a specialized **Financial Retrieval-Augmented Generation (RAG) Framework**. Unlike conventional static banking dashboards or generic chatbots that hallucinate numbers, this system grounds every metric, financial health score, chart visualization, and AI Advisor recommendation strictly in the user's real banking data and uploaded statements (PDF , CSV , XLSX , DOCX , or TXT).



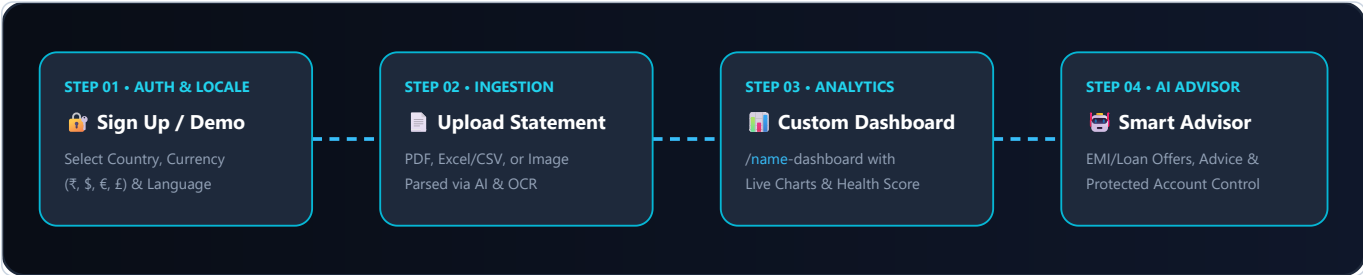
Core Technology Stack

Layer	Technology	Role in the Architecture
Frontend UI	React 18, Vite, Tailwind CSS, Recharts, Lucide	Personalized <code>/<username>-dashboard</code> routing, dynamic KPI cards, interactive charts, and real-time AI Advisor chat
Backend API	Python 3.11, FastAPI, Uvicorn, SQLAlchemy	REST API endpoints, statement ingestion pipeline, financial health engine, and RAG context synthesis
Document Parser	<code>pypdf</code> , <code>python-docx</code> , <code>openpyxl</code> , Custom Regex	Multi-format bank statement parser supporting standard and Indian (<code>INR</code> / <code>Cr</code> / <code>Dr</code>) statement layouts
AI & RAG Engine	Google Gemini (<code>gemini-2.0-flash</code>), Custom RAG	Real-time retrieval of ledger aggregates, category breakdowns, and rule-based guardrails into structured prompts
Database	PostgreSQL (Production) / SQLite (Local Dev)	Relational storage for <code>User</code> , <code>Account</code> , <code>Transaction</code> , <code>Budget</code> , and <code>Insight</code> entities
Cloud Hosting	Render Cloud (Static Site + Web Service)	Automated CI/CD deployment from GitHub (<code>origin/main</code>), portable to any Linux VM or Docker host

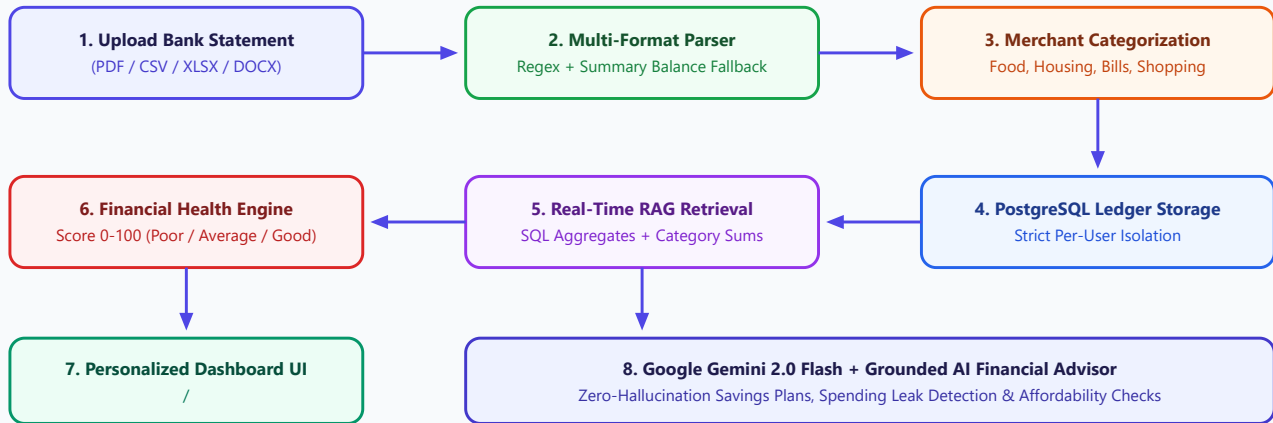


2. Deep-Dive: How the Financial RAG Framework Works Step-by-Step

Retrieval-Augmented Generation (RAG) bridges the gap between a user's private financial ledger and a Large Language Model (Google Gemini). Rather than training a model on static historical data, the platform dynamically parses, indexes, retrieves, and injects exact mathematical aggregates into every dashboard view and AI prompt in real time.



Financial RAG End-to-End Pipeline Architecture



Step 1: Multi-Format Document Ingestion (`POST /users/{id}/upload-statement`)

When a user signs up or uploads a bank statement on the platform, the file is transmitted via `multipart/form-data` to `upload_statement` in `backend/app/main.py` and processed by `parse_statement_bytes` in `backend/app/services/statement_parser.py` : - **CSV / TXT Files**: Decoded using UTF-8/Latin-1 fallback and inspected via `csv.DictReader`. Supports columns such as `Date`, `Description / Narration / Particulars`, `Amount`, `Type`, `Debit / Withdrawal`, `Credit / Deposit`, and `Balance`. - **Excel Spreadsheets (.xlsx)**: Loaded in read-only mode via `openpyxl` and mapped through header normalization. - **Word Documents (.docx)**: Extracted across both paragraph blocks and embedded tables via `python-docx`. - **PDF Bank Statements (.pdf)**: Extracted page-by-page using `pypdf.PdfReader` and passed to the multi-tier regex text parser `_parse_free_text`.

Step 2: Intelligent Regex & Summary Extraction (Zero-Transaction PDF Support)

Real-world bank statements vary wildly in layout. Our parser implements a **two-stage extraction mechanism**: 1. **Line-Level Transaction Extraction**: Matches date patterns (`DD-MM-YYYY`, `YYYY-MM-DD`, `DD Mon YYYY`), strips reference codes, identifies `Cr / Dr` or `+ / -` indicators, and extracts individual debit and credit transactions. 2. **Summary-Level Fallback Extraction**: Many bank statements (such as newly opened accounts or summary certificates) have **zero transaction rows** (`No transactions found`) and only display summary headers (`Opening Balance: INR 0.00`, `Closing Balance: INR 0.00`, `Total Credit Amount: 0.00`, `Total Debit Amount: 0.00`). - If no individual transaction lines exist, the parser scans the document for **Closing Balance**, **Opening Balance**, **Total Credit Amount**, and **Total Debit Amount** using specialized regular expressions. - If the statement explicitly shows `0.00` balance and `0` transactions, the system records `parsed_balance = 0.0` and sets `statement_uploaded = True` on the user's account so the dashboard reflects the exact `0.00` state of the uploaded PDF rather than falling back to placeholder data.

Step 3: Semantic Merchant Categorization

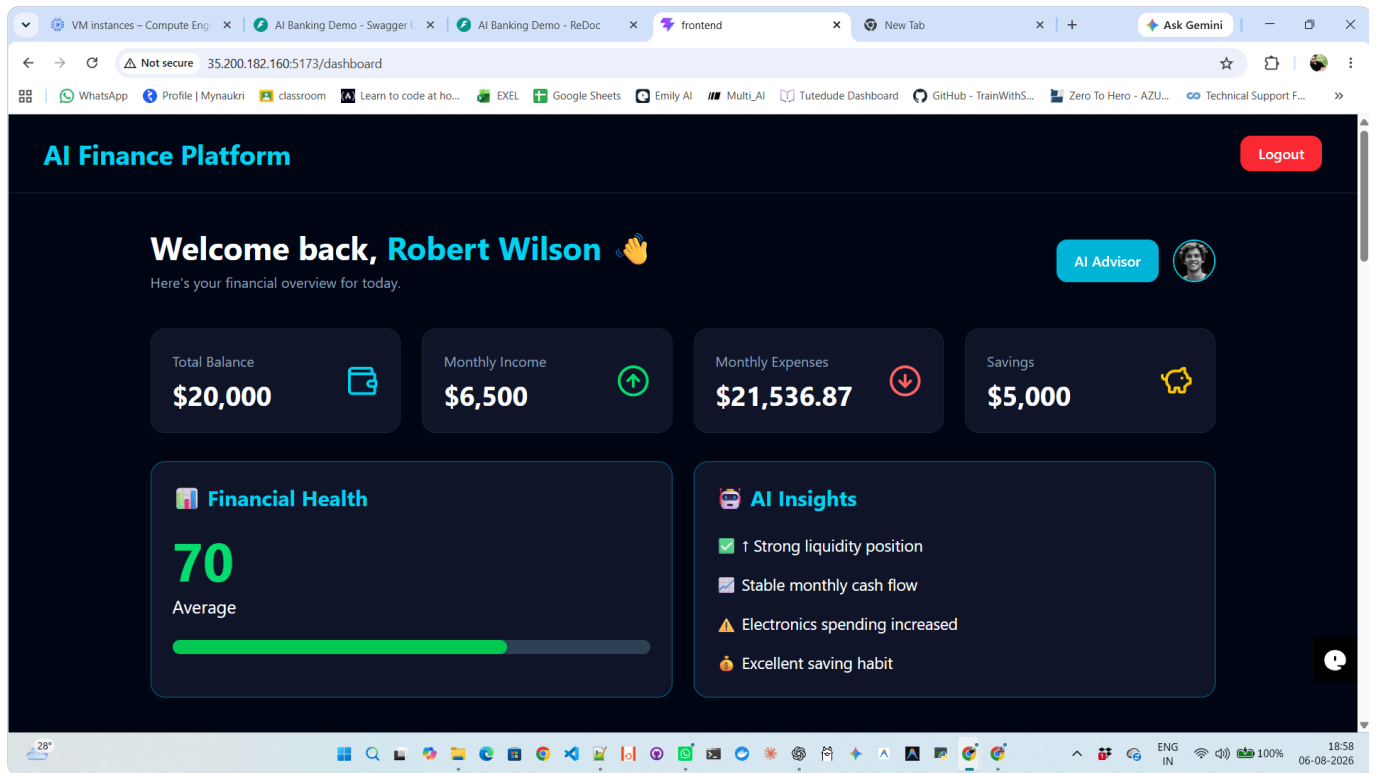
Each extracted transaction description is classified by `_guess_category` into standardized financial buckets: - **Income**: salary, payroll, bonus, dividend, refund, interest - **Housing**: rent, mortgage, hoa, apartment, maintenance - **Food**: grocery, restaurant, swiggy, zomato, starbucks, cafe, supermarket - **Transport**: uber, ola, lyft, metro, fuel, petrol, parking, airline - **Utilities**: electric, water, internet, broadband, phone, mobile, gas bill - **Entertainment**: netflix, spotify, prime, cinema, movie, gaming - **Shopping**: amazon, flipkart, myntra, walmart, target, mall - **Healthcare**: hospital, pharmacy, doctor, medical, clinic, apollo

Step 4: Relational Indexing & Strict Data Isolation

All parsed records are committed to PostgreSQL under the user's `Account` (`user_id`) and `Transaction` (`account_id`) tables. - **Strict Per-User Isolation**: Every API endpoint filters strictly by `user_id`. A newly created account never inherits demo transactions (`Robert Wilson`). - **Statement-Driven State Tracking**: The `Account` model tracks whether a user has uploaded a bank statement (`statement_type` / `statement_uploaded`), ensuring that a user who uploads an empty/ `$0` statement sees `$0` across all KPI cards, charts, and AI Spending Insights.

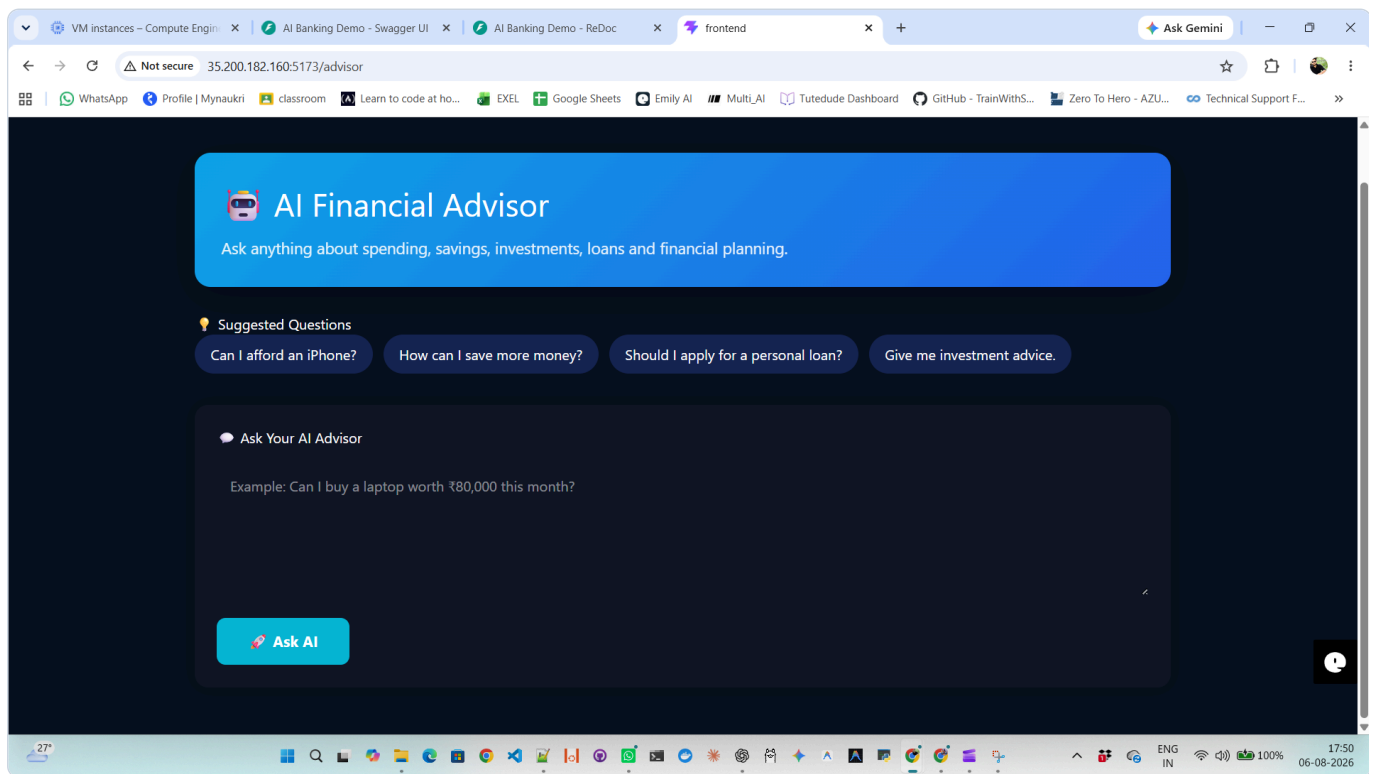
Step 5: Real-Time Retrieval & Financial Health Scoring

Whenever the user opens their dashboard or sends a message to the AI Advisor, the backend executes **Step R (Retrieval)** of the RAG pipeline: 1. `get_financial_summary` retrieves the user's exact `total_balance`, `monthly_income`, `monthly_expenses`, `savings`, and category totals from PostgreSQL. 2. `calculate_health_score` computes a deterministic **Financial Health Score (0-100)**: - **Zero-Liquidity Guardrail**: If `total_balance <= 0` and `monthly_income <= 0` (for example, when a user uploads a bank statement with `$0` balance and `$0` savings), the engine immediately assigns a score of `20 / 100` with status `Poor`, alerting the user that their account has zero active liquidity. - **Active Cash-Flow Scoring**: When income is present, the score is weighted across **Savings Rate (0-40 pts)**, **Expense Ratio (0-40 pts)**, and **Liquidity Buffer (0-20 pts)**, classifying health into `Good ( >= 75 )`, `Average ( 45-74 )`, or `Poor ( < 45 )`.



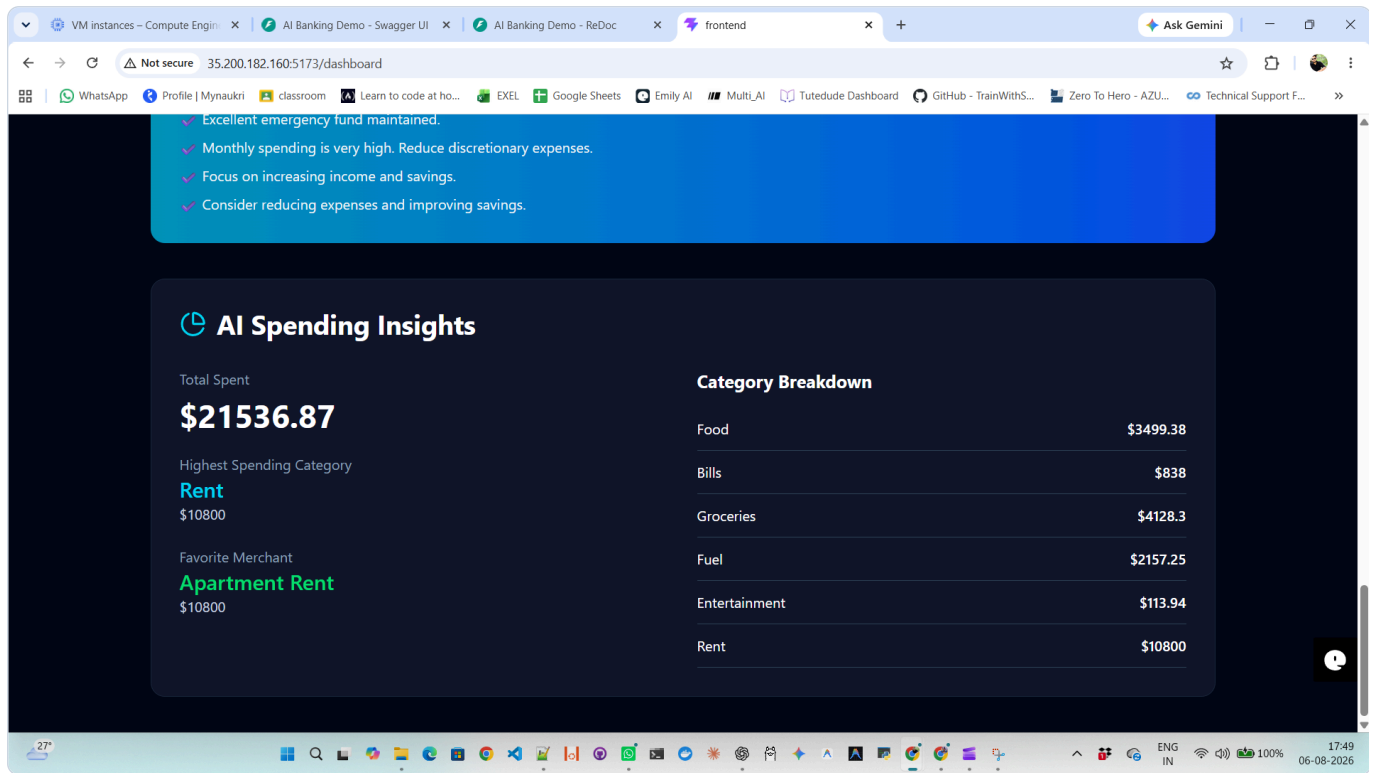
Step 6: Context Augmentation & Grounded LLM Generation

In `generate_financial_advice` (`backend/app/ai/advisor.py`), the retrieved financial snapshot is serialized into a structured system prompt and sent to **Google Gemini** (`gemini-2.0-flash`): - The prompt includes exact **Total Balance**, **Monthly Income**, **Monthly Expenses**, **Net Monthly Savings**, **Savings Rate (%)**, and **Category Breakdown**. - If the Gemini API key is rate-limited or offline, the system seamlessly falls back to a deterministic, math-verified RAG rule engine so the user always receives instantaneous, accurate financial advice.



3. Benefits of the RAG Framework in Savings & Expenditure Management

Traditional finance apps only show historical tables of where money *went*. By combining deterministic SQL aggregation with LLM reasoning via RAG, this framework actively helps users **reduce unnecessary expenditure** and **accelerate wealth & savings**:



1. Zero-Hallucination Expenditure Tracking

Generic AI models guess numbers or give vague textbook advice. Because our RAG pipeline retrieves exact category sums from the user's parsed bank statement before generating a response, every insight cites the user's actual rupee/dollar figures, merchant categories, and exact percentages of total spend.

2. Automated Detection of "Spending Leaks"

In `get_spending_insights`, the RAG analytical layer ranks every expense bucket by percentage of total expenditure: - **High-Concentration Alerts ($\geq 30\%$ of spend)**: Automatically flags dominant cost centers (e.g., Housing, Shopping, or Dining Out) and calculates the exact monthly dollar amount saved if the user trims that category by **10% to 15%**. - **Secondary Category Optimization ($\geq 15\%$ of spend)**: Highlights discretionary categories (such as Entertainment, Food delivery, or Subscriptions) where automated monthly budget caps can stop lifestyle creep.

3. Dynamic 50/30/20 Savings Rate Optimization

The framework continuously benchmarks the user's real cash flow against the gold-standard **50/30/20 Financial Rule** (50% Needs, 30% Wants, 20% Savings & Debt Repayment): - **When Savings Rate $< 20\%$** : The AI Advisor calculates the exact monthly dollar gap required to reach the 20% threshold and recommends an automatic payday transfer for that exact amount. - **When Savings Rate $\geq 20\%$** : The AI Advisor calculates how much surplus cash sits above a 3-to-6-month emergency fund and suggests deploying that surplus into diversified index funds (`S&P 500` / `Nifty 50`) or high-yield instruments.

4. Pre-Purchase Affordability Simulation ("Can I Afford This?")

Users can ask the AI Advisor natural-language questions such as: - "Can I afford a \$3,000 vacation next month?" - "How can I save \$500 more per month?" - "Where am I overspending right now?"

The RAG engine extracts the target purchase price from the user's query, compares it against both **Net Monthly Cash Flow (`Income - Expenses`)** and **Total Liquid Balance**, and tells the user immediately whether they can pay out of monthly surplus, how many months they need to save for it, or what percentage of their liquid net worth it would consume.

5. Immediate Zero-Balance & Overdraft Risk Detection

If a user uploads a bank statement with **\$0 Balance and \$0 Savings** (or negative cash flow where `Expenses > Income`), the RAG framework immediately switches the Financial Health card to **Poor (`20/100`)** and generates critical liquidity alerts in both the Dashboard and AI Spending Insights—preventing overdraft fees and guiding the user to establish an initial emergency buffer.

4. Step-by-Step Process Execution on Render Cloud

The production platform runs live on **Render Cloud** across a decoupled two-service architecture connected to a managed PostgreSQL database: -

Frontend Static Site: <https://ai-banking-system-1-pdow.onrender.com> - **Backend Web Service:** <https://ai-banking-system-3gzg.onrender.com>

Process 4.1: Automated Build & Startup Lifecycle on Render

1. **GitHub Push Trigger:** Pushing a commit to `origin/main` triggers automatic builds on both Render services.
2. **Backend Startup (`backend/app/main.py`):**
3. Render runs `uvicorn app.main:app --host 0.0.0.0 --port $PORT`.
4. On `@app.on_event("startup")`, FastAPI executes `Base.metadata.create_all(bind=engine)` and runs non-destructive schema migrations (`ALTER TABLE users ADD COLUMN IF NOT EXISTS password_hash`, etc.).
5. Next, `seed_database()` ensures the default Demo User (**Robert Wilson**, `demo@aibanking.com`, `id=1`) exists with his baseline `$20,000` balance, `$8,500` monthly income, `$3,350` expenses, and `$5,150` savings.
6. **Frontend Build & SPA Routing (`frontend/public/_redirects`):**
7. Render builds the Vite bundle via `npm install && npm run build` and publishes the `dist/` directory.
8. The `/* /index.html 200` rule inside `frontend/public/_redirects` ensures deep URLs like `/aaditya-dashboard` or `/robert-dashboard/ai-advisor` resolve directly to React Router without `404` errors.

Process 4.2: Dynamic Personalized URL Routing (`/<firstname>-dashboard`)

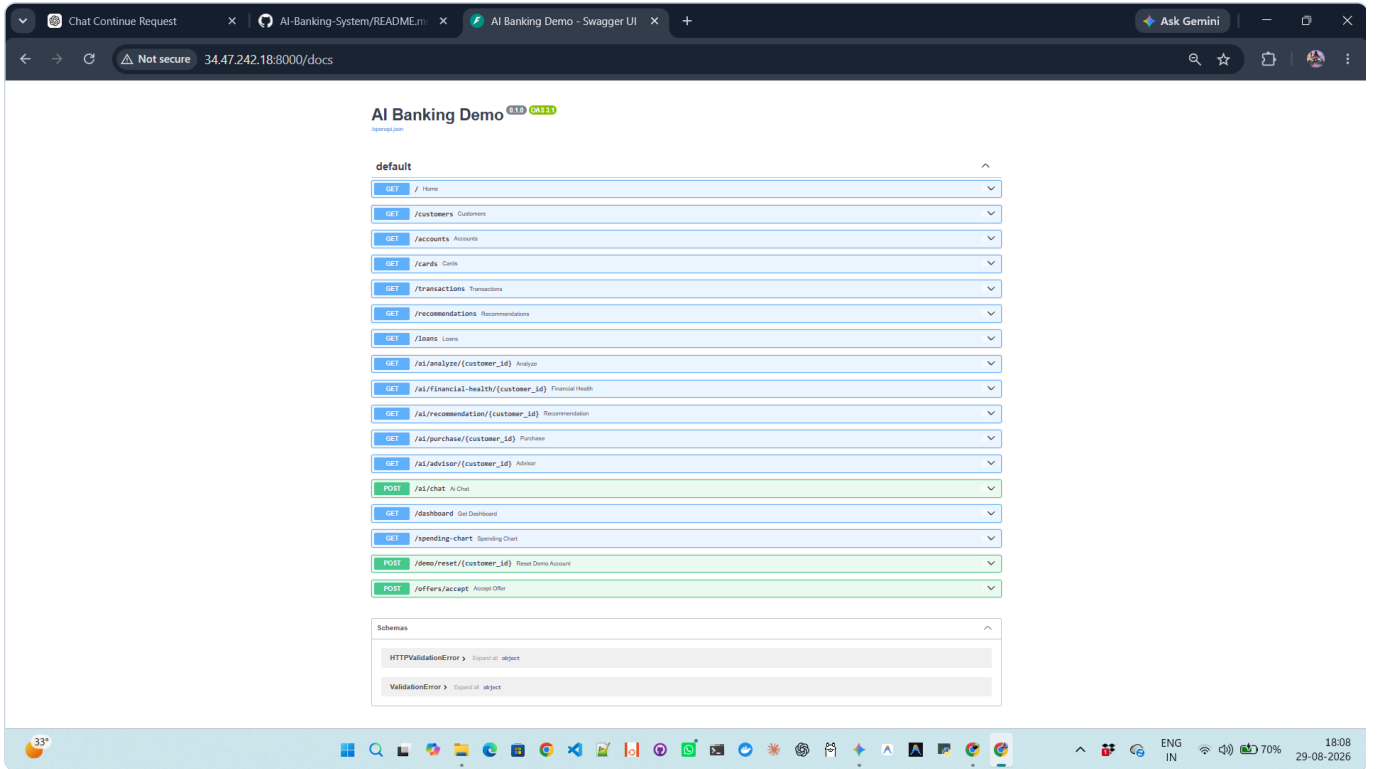
- When any user logs in or registers (e.g., `Aaditya`), `getUserSlug` extracts the lowercase first name (`aaditya`) and routes the browser to:
- `/aaditya-dashboard` (Main Financial Dashboard)
- `/aaditya-dashboard/transactions` (Ledger & Statement Manager)
- `/aaditya-dashboard/analytics` (Category & Cash Flow Analytics)
- `/aaditya-dashboard/ai-advisor` (Conversational RAG Financial Advisor)
- When a page loads, `App.jsx` calls `GET /users/by-slug/{slug}` on the backend to resolve the matching user account and hydrate the session.

Process 4.3: Demo Account Auto-Restore on Hard Refresh (`F5`)

- Clicking **Live Demo** calls `POST /auth/demo-login`, logging right into **Robert Wilson** (`/robert-dashboard`).
- If a visitor clicks **Clear Data** inside the demo account, the balance drops to `$0`.
- However, pressing **F5 (Hard Refresh)** in the browser triggers `isBrowserReload()` via the browser's `performance.getEntriesByType('navigation')` API, which automatically invokes `POST /users/1/reset-demo` on the backend—instantly restoring Robert Wilson's `$20,000.00` balance, `$8,500.00` income, `$3,350.00` expenses, and `$5,150.00` savings before the dashboard renders.

Process 4.4: 3-Stage Conversational & UI Account Deletion Guardrails

- **UI Modal Deletion:** Clicking **Delete Account** in the top navigation bar prompts the user for their login password, verifies it via `POST /users/{id}/delete-account`, purges all user data, clears `localStorage`, and redirects to `/login?mode=register`.
- **Conversational AI Deletion:** If a user tells the AI Advisor *"Delete my account"*:
- **Stage 1 (Retention Persuasion):** The AI Advisor highlights their financial tools and asks if they are certain they still want to delete their account.
- **Stage 2 (Password Prompt):** If the user insists (*"Yes, delete my account"*), the AI Advisor requests their password in the next chat reply.
- **Stage 3 (Verified Purge & Redirect):** Once the password is sent in chat, the backend verifies it, deletes the account and all cascading records, and the frontend automatically redirects to the **Create Account** screen after 1.5 seconds.
- **Demo Account Protection (`robert@123`):** Nobody can delete Robert Wilson's demo account without the master admin password `robert@123`. Even if deleted by an admin, the demo account immediately re-seeds itself so the Live Demo button never breaks for future visitors.



5. Step-by-Step Guide: Switching from Render to Any Other Virtual Machine (VM)

The entire platform is built on standard open-source components (**Python FastAPI**, **React/Vite**, and **PostgreSQL**) with zero vendor lock-in. You can migrate the system from Render to **AWS EC2**, **Microsoft Azure VM**, **Google Cloud Compute Engine (GCE)**, **DigitalOcean Droplet**, **Hostinger VPS**, or an **on-premise Ubuntu Server** in under 10 minutes by changing **only 2 environment files**.

5.1 The Only Configuration Variables You Need to Change

Variable Name	File / Location	Current Render Value	New VM Value
DATABASE_URL	backend/.env	postgresql://...@dpg-...render.com/db	postgresql://bankuser:bankpass@localhost:5432/aibanking
GEMINI_API_KEY	backend/.env	AIzaSy...	AIzaSy... (Your Google Gemini API Key)
VITE_API_URL	frontend/.env	https://ai-banking-system-3gzg.onrender.com	http://<YOUR_VM_IP_OR_DOMAIN>:8000 (or /api behind Nginx)

5.2 Option A: 1-Command Migration to Any VM Using Docker Compose (Recommended)

A `docker-compose.yml` file is already included in the repository root.

Step 1: Provision a Linux VM & Install Docker

SSH into your new Ubuntu 22.04 / 24.04 LTS VM (AWS EC2, DigitalOcean, Azure, GCP) and run:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl docker.io docker-compose-plugin
sudo systemctl enable --now docker
```

Step 2: Clone the Repository

```
git clone https://github.com/ai-aadi-ops/AI-Banking-System.git
cd AI-Banking-System
```

Step 3: Set Your VM Environment Variables

Create a `.env` file in the root directory (or update `docker-compose.yml`):

```
GEMINI_API_KEY=your_actual_gemini_api_key_here
VITE_API_URL=http://<YOUR_VM_PUBLIC_IP>:8000
```

Step 4: Launch the Entire Stack

```
sudo docker compose up -d --build
```

- **PostgreSQL 15** starts on internal port `5432` with persistent volume storage.
- **FastAPI Backend** starts on port `8000` (`http://<YOUR_VM_PUBLIC_IP>:8000/docs`).
- **React Frontend** starts on port `5173` (`http://<YOUR_VM_PUBLIC_IP>:5173`).

5.3 Option B: Native Ubuntu VM Deployment (`Systemd` + `Nginx` + `PostgreSQL`)

If you prefer running directly on a bare-metal or cloud Ubuntu VM without Docker, follow these 5 steps:

Step 1: Install System Packages on Ubuntu VM

```
sudo apt update && sudo apt install -y python3-pip python3-venv nodejs npm postgresql postgresql-contrib nginx git
```

Step 2: Configure PostgreSQL Database

```
sudo -u postgres psql -c "CREATE USER bankuser WITH PASSWORD 'StrongPassword123';"
sudo -u postgres psql -c "CREATE DATABASE aibanking OWNER bankuser;"
```

Step 3: Set Up the FastAPI Backend & `systemd` Daemon

```
cd /var/www
sudo git clone https://github.com/ai-aadi-ops/AI-Banking-System.git
cd /var/www/AI-Banking-System/backend

python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

cat <<EOF > .env
DATABASE_URL=postgresql://bankuser:StrongPassword123@localhost:5432/aibanking
GEMINI_API_KEY=your_gemini_api_key_here
EOF
```

Create `/etc/systemd/system/ai-banking-backend.service` :

```
[Unit]
Description=AI Banking FastAPI Backend
After=network.target postgresql.service

[Service]
User=www-data
WorkingDirectory=/var/www/AI-Banking-System/backend
EnvironmentFile=/var/www/AI-Banking-System/backend/.env
ExecStart=/var/www/AI-Banking-System/backend/venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8000 --workers 2
Restart=always

[Install]
WantedBy=multi-user.target
```

Enable and start the backend service:

```
sudo systemctl daemon-reload
sudo systemctl enable --now ai-banking-backend
```

Step 4: Build the React Frontend for Your VM Domain / IP

```
cd /var/www/AI-Banking-System/frontend
echo "VITE_API_URL=http://<YOUR_VM_IP_OR_DOMAIN>/api" > .env
npm install
npm run build
```

Step 5: Configure Nginx Reverse Proxy & SPA Routing

Create `/etc/nginx/sites-available/ai-banking` :

```
server {
    listen 80;
    server_name <YOUR_VM_IP_OR_DOMAIN>;

    # Serve React Frontend Static Build
    root /var/www/AI-Banking-System/frontend/dist;
    index index.html;

    # SPA Deep-Link Routing (/<username>-dashboard, /login, etc.)
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Reverse Proxy Backend API Requests to FastAPI (Port 8000)
    location /api/ {
        rewrite ^/api/(.*)$ /$1 break;
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        client_max_body_size 25M;
    }
}
```

Enable the site and restart Nginx:

```
sudo ln -s /etc/nginx/sites-available/ai-banking /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl restart nginx
```

To enable free HTTPS (SSL/TLS) on your custom domain on the VM, simply run: `sudo apt install -y certbot python3-certbot-nginx && sudo certbot --nginx -d yourdomain.com`